BOB-112 â€” Live wrk Load-Test Verification: /healthz TTL Cache

Revision: 1 **Last modified:** 2026-08-18T19:20:00Z

What this closes

BOB-112 (`qBitTorrent-go/internal/jackettapi/health.go`, commit `91b52db`) added a 30s TTL cache around `HandleHealth`â€™s Jackett liveness check to stop `/healthz` from making a synchronous, uncached `Jackett.GetCatalog()` call on every single request â€” a self-inflicted DDoS amplification vector discovered by `challenges/scripts/ddos_resilience_challenge.sh` (see `docs/testing/ddos_resilience.md` â€œFindingsâ€ #2). That commitâ€™s own honest declaration was: *â€œLive wrk load-test tracked as followupâ€*. This document is that followup â€” real, captured, Â§11.4.6-compliant `wrk` evidence with Â§11.4.115 RED/GREEN polarity, run live against the rebuilt container on 2026-08-18.

Method

1. Rebuilt + force-recreated the `boba-jackett` container from the committed `91b52db` source (`podman-compose build boba-jackett` + `podman-compose up -d --force-recreate boba-jackett`) â€” the running container had previously been serving an image built **2026-08-08**, ten days before the BOB-112 fix landed, so it was NOT exercising the fix without this rebuild. Confirmed via the new, fix-only log line `/healthz Jackett cache refresh #1 (hits=0 misses=1 ...)`.
2. **Post-fix (GREEN) run**: `wrk -t4 -c100 -d30s --timeout 3s --latency http://localhost:7189/healthz` against the rebuilt container running the real, committed BOB-112 code. Verbatim output: `docs/qa/BOB-112/wrk_evidence.log`.
3. **Â§1.1 mutation (RED baseline)**: `jackettOk()` in `health.go` was edited (uncommitted, in-place) to bypass the cache *and* `refreshJackettOk()`â€™s own double-checked-locking entirely, calling `d.Jackett.GetCatalog()` directly on every hit â€” reproducing the exact pre-BOB-112 code shown in `docs/testing/ddos_resilience.md` â€œFindingsâ€ #2 (`if d.Jackett != nil { _, err := d.Jackett.GetCatalog(); ... }`). Container rebuilt + recreated with the mutated source, then the identical `wrk` load re-run. Verbatim output: `docs/qa/BOB-112/wrk_mutation_baseline.log`.
4. Mutation reverted from a pre-edit backup; `git diff --stat` on `health.go` confirmed **empty** (byte-identical to HEAD) before rebuilding a third time. The rebuildâ€™s image digest (`962f4452c725...`) matched the post-fix build from step 2 **exactly**, confirming a clean, complete revert â€” not merely â€œclose enoughâ€. Container force-

recreated a final time back onto this image; `/healthz` verified `HTTP 200` afterward.

Load parameters throughout: 4 threads x 100 connections x 30s wall-clock, localhost-only, bounded per Â§12.6/Â§12.11 (client-side `wrk` process on an 8-core/62GiB host with 51GiB free at test time â€” no host-safety incident).

Results

Metric	Baseline (mutation, RED â€” cache bypassed)	Post-fix (GREEN â€” real BOB-112 code)
Total requests (30s)	412	812,149
Timeouts (`wrk` socket-error `timeout`)	**400 / 412 = 97.1%**	**0 / 812,149 = 0.0%** (no `Socket errors` line printed â€” wrk omits it when all three counters are 0)
Requests/sec	13.71	27,049.00
Avg latency	1.87s	4.79ms
p50 latency	1.97s	3.40ms
p90 latency	2.37s	9.34ms
p99 latency	2.67s	19.89ms
Server-side upstream `Jackett.GetCatalog()` calls (from `podman logs boba-jackett`)	every one of the 412 requests (0 `cache refresh` log lines â€” the mutated path never reaches `refreshJackettOk()`, which is the only place that log line is emitted, confirming zero caching occurred)	2 (`cache refresh #1` at boot, `#2` ~39s later once the 30s TTL first expired mid-run) across the entire 812,149-request run

Improvement

- **Timeout rate: 97.1% -> 0.0%** â€” the cache is load-bearing; without it, under this load, `/healthz` fails for the overwhelming majority of callers.
- **Throughput: 13.71 req/s -> 27,049.00 req/s** (~1,973x more requests served in the same 30s window).
- **Avg latency: 1.87s -> 4.79ms** (~390x faster).
- **p99 latency: 2.67s -> 19.89ms** â€” the mutationâ€™s p99 sits right at the 3s client-side `--timeout` budget (consistent with `docs/testing/ddos_resilience.md`â€™s â€œqueued behind Jackett round-tripsâ€¦ description); the fixâ€™s p99 stays two orders of magnitude below that budget.

- **Upstream Jackett load: ~412 calls/30s -> 2 calls/30s** â€” the cache collapses what would have been one upstream call per request into effectively one call per TTL window, which is the mechanism behind every other number in this table.

Anti-bluff notes (Â§11.4.6 / Â§11.4.115)

- Every number above is read directly from the two `wrk_*.log` files committed alongside this summary â€” nothing paraphrased or estimated.
- The RED/GREEN polarity holds by construction: the mutation reproduces the documented pre-fix defect (verified independently via the log-based â€œzero cache-refresh linesâ€ check, not merely the timeout numbers), and the revert is verified byte-identical via both `git diff --stat` (empty) and matching container image digest before the final rebuild â€” not just â€œit looked the sameâ€.
- The container the mutation ran against was confirmed to be genuinely running the mutated code (not a stale image) via its own fresh `StartedAt` timestamp and image digest after each `--force-recreate`.
- `docs/qa/BOB-112/wrk_evidence.log` and `wrk_mutation_baseline.log` are the raw captured `wrk` stdout, unedited.

BOB-113 side effect

Running `bash scripts/install-dev-tools.sh` (a prerequisite for this taskâ€™s `wrk` load test) also produced the BOB-113 verification evidence â€” see `.superpowers/sdd/task-74-report.md` for the captured `wrk --version` output and install-script summary (`wrk`/`hey` `ALREADY-PRESENT`, `siege` honest `SKIP` with the documented manual sudo command, `ab`/`curl-loader` detection only). No separate doc was created for BOB-113 per the task briefâ€™s â€œside effectâ€ framing â€” the evidence is captured in the task report and in this sessionâ€™s terminal history.